

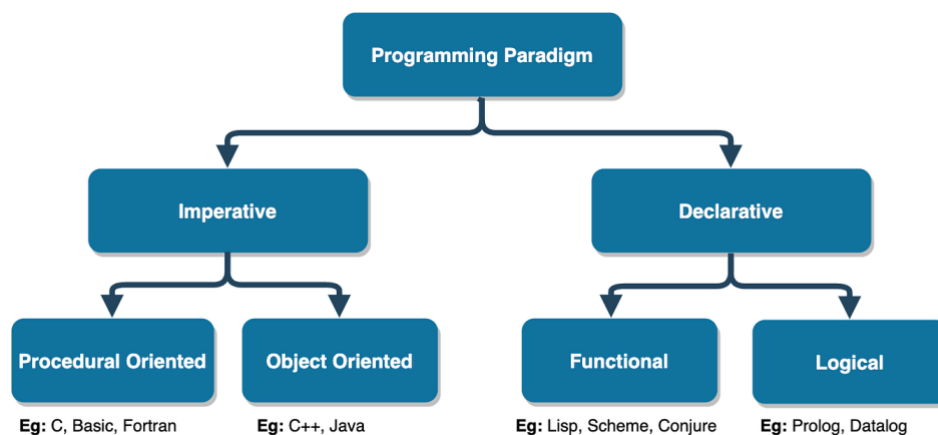
I paradigmi di programmazione

Un paradigma di programmazione è una visione concettuale, un approccio, un modo di pensare alla costruzione del software. Non si tratta semplicemente di un insieme di regole tecniche, ma di una "filosofia": definisce come un programmatore concepisce il problema da risolvere, come organizza la logica, come gestisce i dati e come controlla il flusso di esecuzione di un programma.

Esistono diversi paradigmi, ognuno con le proprie caratteristiche, vantaggi e limiti, nati e sviluppatisi nel tempo in risposta a esigenze pratiche: dalla gestione della complessità dei sistemi, alla necessità di riuso del codice, fino alla scrittura di software affidabile, efficiente o scalabile.

Ogni linguaggio di programmazione, in genere, si ispira ad un particolare paradigma (a volte anche più di uno).

Una prima grande distinzione da fare è quella tra **paradigmi imperativi** e **paradigmi dichiarativi**.



Nel paradigma imperativo, il programmatore dice **come** deve essere fatto qualcosa. Il programma è una sequenza di istruzioni, dette anche **direttive** o **comandi**, che modificano lo stato del sistema passo dopo passo. È un approccio simile a dare ordini: fai questo, poi fai quello, poi cambia quel valore, e così via. Il paradigma imperativo è il più antico e tradizionale, ed è alla base di linguaggi come il C o il Pascal.

Esempi		
$x = x + 1$	$media = somma / n$	$a[i] = rand() \% 10$

Una sua variante strutturata è il **paradigma procedurale**, in cui il codice viene suddiviso in procedure o funzioni, per migliorarne la modularità. Tuttavia, entrambi i paradigmi rimangono strettamente legati all'idea di controllo diretto del flusso e della memoria.

Diverso è invece l'approccio dichiarativo, dove il programmatore si concentra non su come ottenere il risultato, ma su **cosa** vuole ottenere. È come dichiarare un obiettivo, lasciando al sistema il compito di capire il percorso migliore per raggiungerlo. Questo approccio è più vicino alla logica, all'astrazione, e in molti casi alla matematica. I programmi non sono più una sequenza di "ordini", ma più una sequenza di "affermazioni", che la macchina virtuale del linguaggio ritiene o deve rendere vere.

Esempio

```
SELECT * from studenti where eta > 18
```

All'interno dell'universo dichiarativo troviamo, ad esempio, la **programmazione logica**, dove si definiscono fatti e regole, e il programma consiste in interrogazioni a un sistema che deduce le risposte (come in Prolog). Un altro paradigma dichiarativo è la **programmazione funzionale**, basata sul concetto di funzioni pure, senza effetti collaterali, e sull'immutabilità dei dati. In questo modello, il programma è una composizione di funzioni matematiche, spesso molto compatto e concettualmente elegante. Haskell è uno dei suoi esempi più noti.

Esempio

```
map(f, lista)
```

A metà strada fra il dichiarativo e l'imperativo si colloca il paradigma probabilmente più diffuso nel software moderno: la **programmazione orientata agli oggetti (OOP)**. In questo modello, si cerca di rappresentare entità del mondo reale attraverso

oggetti, che racchiudono al loro interno sia dati (stato) che comportamenti (metodi). Gli oggetti interagiscono tra loro tramite messaggi e la struttura del programma riflette spesso la struttura del dominio applicativo. Concetti come ereditarietà e polimorfismo permettono di estendere e riutilizzare il codice in modo modulare.

Vale la pena sottolineare che la maggior parte dei linguaggi moderni non aderisce rigidamente a un solo paradigma, ma sono **multiparadigma**: permettono, e spesso incoraggiano, l'uso di più stili nello stesso programma. Python, per esempio, consente uno stile procedurale, ma anche funzionale e orientato agli oggetti; JavaScript può essere usato in modo funzionale, imperativo o asincrono.

In conclusione, i paradigmi di programmazione non sono solo strumenti tecnici, ma modelli mentali che influenzano il modo in cui pensiamo ai problemi. Conoscerli, capirli e saperli combinare in modo critico è una competenza essenziale per ogni programmatore che voglia scrivere software chiaro, efficiente e mantenibile.